Implementation Foundation

Campus Equipment Checkout System | Raghuram Kalasani | COMP-699-A

Step 2, Week 1 - corrected software development milestone, October 27, 2026

Purpose and correction

This milestone documents the initial executable software foundation for the software development project established in Step 1. The earlier research_foundation report followed the research-project instructions by mistake. This report supersedes that document for Week 1. The earlier research material is retained as historical work and is not evidence that the application requirements are complete.

Implemented source tree

```
src/campus_checkout/
  manage.py                 Django command entry point
  requirements.txt          Pinned application dependencies
  config/
    settings.py             Apps, SQLite, middleware, timezone
    urls.py                 Root redirect and admin route
    asgi.py, wsgi.py         Server entry points
  inventory/
    models.py               Equipment data model
    admin.py                Inventory interface and controls
    migrations/0001_initial.py
    tests.py                Six automated test methods
```

Components and interaction

The browser uses Django admin at /admin/. Django authentication and permissions protect that interface. Inventory forms validate input and use the Equipment model through the Django ORM. SQLite stores the equipment records. The root URL redirects to /admin/. The application is a local classroom prototype with a working inventory interface, not the complete lending system.

Equipment has a unique asset_tag, name, category, optional description, and is_active flag. Asset tags accept uppercase letters, digits, and hyphens. The database enforces uniqueness. The admin supports search and filtering, creation, editing, and retirement by clearing Active. Admin deletion is disabled to preserve records for future history.

Installation and execution

Run these commands from the repository root using Python 3.12. The tested environment used Python 3.12.14, Django 5.2.18, SQLite 3.53.1, and macOS 15.6.1 on arm64. Dependencies are pinned in src/campus_checkout/requirements.txt. Python 3.9 from the Mac system installation is not the interpreter used for this Django application.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r src/campus_checkout/requirements.txt
cd src/campus_checkout
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Choose a local demonstration username and password when prompted. No live account or database is included. If python3.12 is unavailable, use the path to an installed Python 3.12 interpreter for the first command. A previously working Python 3.12 virtual environment can also be reused after checking python --version.

Manual inventory demonstration

1. Open http://127.0.0.1:8000/admin/ and sign in with the account you created. 2. Add equipment with asset tag LAP-001, name Demo laptop, category Laptop, and Active selected. 3. Save, search for LAP-001, and edit its description. 4. Clear Active and save to retire the item. 5. Try adding another item with LAP-001 and confirm the duplicate is rejected. Use a fresh database or a different tag if LAP-001 already exists.

The expected result is a stored inventory record that can be retrieved and edited. Retirement changes the flag and preserves the record. This demonstrates inventory persistence, not reservation availability or checkout processing.

Running checks

```
# Stop runserver with Control+C before using this terminal.
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test inventory --verbosity 2
```

These commands assume you remain in src/campus_checkout with the virtual environment active. On another day, activate .venv from the repository root before changing into that directory. A running development server occupies its terminal; do not type test commands into its log output.

Recorded execution and evidence

The assistant reran the copied application on October 27, 2026. The system check reported no issues, the migration check reported no changes, fresh database migrations completed, and all six inventory tests passed. This is an assistant-run verification record, not a claim that the student independently reran these checks on October 27. The student previously showed the running inventory interface on October 17.

Unedited captured command results are under docs/step_2/week_1/verification/: system_check.txt, migration_check.txt, database_setup.txt, test_output.txt, and environment.txt. Each command log records its command and exit status. The verification database was excluded from the upload package.

What the tests verify

Database uniqueness: a duplicate asset tag raises IntegrityError. Input validation: an invalid tag fails model validation. Anonymous access: the equipment list redirects to login. Borrower access: a direct add request does not create an item. Staff access: staff without model permissions cannot open the add form. Administrator workflow: creation and editing persist, retirement clears Active, and a delete request is denied while the record remains.

Status against the original requirements

FR-01 - partially implemented. Django admin authentication is available. A borrower login interface and complete login/logout acceptance coverage remain incomplete.

FR-02 - partially implemented. Admin endpoints enforce authentication and model permissions. Borrower ownership rules and reservation authorization are not implemented.

FR-03 - partially implemented. Administrators can create and update inventory with unique identifiers. The inactive-item reservation restriction remains unimplemented because reservations are not integrated.

FR-04 - planned for borrowers. Admin inventory search exists, but borrower catalog access and time-interval availability checks do not.

FR-05 through FR-12 - planned in the application. The earlier standalone reservation proof of concept remains separate. It does not demonstrate integrated approval, cancellation, checkout, return, overdue reporting, or reservation/loan persistence in this application.

Design decisions and limitations

The implementation retains the Step 1 Django and SQLite architecture. Django admin provides the first inventory interface, allowing database and authorization behavior to be exercised before custom borrower pages exist. This is an implementation sequence decision, not a change from a software development project to a research project.

Only the Equipment domain model is implemented in this application. Reservations and checkouts from the full design remain future models. The standalone approval proof of concept has not been connected to this application. Deletion is disabled in admin; general database deletion protection for future loan relationships still needs design and implementation.

Configuration uses localhost hosts, DEBUG enabled, America/New_York, and Django CSRF middleware. DJANGO_SECRET_KEY can override the clearly labeled public development placeholder. These are development settings and must not be used for deployment. The package excludes the local SQLite database, virtual environment, caches, and real credentials. Test fixture credentials are synthetic and created only in the test database.

Week 2 implementation plan

Target a coherent subset of FR-02, FR-04, FR-05, and FR-06: add borrower catalog access and server-side permissions; define a Reservation model and validated request form; persist pending requests linked to the signed-in user; and connect administrator approval to a transaction-protected overlap check. Record each requirement as implemented or partial based on actual behavior, not planned behavior.

Add tests for valid requests, invalid time intervals, inactive equipment, unauthorized operations, and conflicting approvals. Keep initial SQLite transaction behavior under test when adapting the earlier proof of concept. Week 3 will integrate a borrower request-to-admin-decision workflow and verify persisted state and meaningful error handling.

Repository correction and provenance

Use docs/step_2/week_1/implementation_foundation.pdf, matching the assignment milestone table and existing repository path. Keep Step 1 submissions unchanged. A new Week 1 README identifies this report as the software development submission and marks research_foundation.md/pdf and their input-generation evidence as superseded. No earlier files need to be deleted or silently renamed.

This correction package was prepared on October 27. Preserve real commit dates and do not imply that this correction was uploaded in earlier weeks. Week 2 and Week 3 reports are not included as completed milestones. ChatGPT assisted with the application, tests, verification, and this report. The student should review the code, reproduce the checks, and explain the demonstrated behavior.

Core Functionality

Scope and development record

This Week 2 milestone implements a reservation subset of the Campus Equipment Checkout software project: FR-02, FR-04, FR-05 and FR-06. It extends the existing Django inventory application rather than the superseded research-input generator. Work and assistant-run verification were performed on October 27, 2026. The milestone title is not a claim that development occurred in an earlier calendar week.

Selected requirements and observable behavior

FR-02: Server-side authentication, reviewer permission and borrower ownership are enforced for implemented routes. Staff status alone does not permit approvals. Borrowers see only their own requests; posted borrower/status fields cannot override the server-selected owner or Pending status. Full-system role coverage remains partial until lending features exist.

FR-04: Authenticated users search active equipment by name or category and enter a future start/end interval. The catalog identifies overlap with approved reservations. Pending requests do not block availability; intervals touching only at an endpoint do not overlap.

FR-05: A valid future request for active equipment creates a persistent Pending reservation linked to the logged-in borrower. Reversed/equal times, past starts, malformed dates, and inactive equipment are rejected. Time controls use America/New_York; timestamps are timezone aware.

FR-06: A user with inventory.review_reservation can approve or reject a pending request. Approval rechecks active equipment, future time, and conflicting approved reservations. Failed approval leaves the request Pending, without reviewer metadata. Repeat decisions are rejected.

Core Functionality / continued

Source modules and component interaction

inventory/models.py and migrations/0002_reservation.py define Reservation, protected user/equipment foreign keys, review metadata, a positive-interval database constraint, and an overlap-query index. Existing inventory records remain intact when the migration is applied.

inventory/forms.py validates browser times. inventory/services.py owns request creation and decision rules. inventory/views.py connects authenticated routes, forms, services and rendered responses; config/urls.py maps them. inventory/templates provides login, catalog, request form, borrower history, and reviewer queue.

POST /equipment/<id>/reserve/ creates requests. GET /reservations/ displays only the caller's records. GET /review/ and POST /review/<id>/ require reviewer permission. GET requests cannot approve or reject; POSTs use Django CSRF middleware. These are HTML routes, not a separate JSON API.

Transaction design and technical problems

SQLite does not implement SELECT FOR UPDATE locking. The application uses short transaction.atomic service calls with SQLite transaction_mode=IMMEDIATE, so writer serialization begins before the conflict read. This avoids a separate check-then-write race. The design follows the documented SQLite transaction option (Django Software Foundation n.d.).

Concurrent tests use a file-backed database and independent connections. An in-memory test database would not represent the intended file-lock behavior. A 20-second busy timeout is configured; the UI displays a retry message if database contention raises OperationalError. This is not a scalability guarantee.

Reservation is deliberately not registered as an editable Django admin model: ordinary UI approvals must pass through the service checks. Direct SQL or privileged Python-shell writes can bypass business rules and are outside the supported application interface.

Core Functionality / continued

Automated tests and measured results

Command: python manage.py test inventory.test_reservations.ReservationTests --noinput --verbosity 2. Result: 14 tests passed. Actual verbose output is retained in week_2/verification/test_output.txt.

Coverage includes persistence of Pending and owner; invalid intervals; inactive equipment; database interval constraint; half-open boundaries; conflict rollback; different equipment; rejection and repeated decisions; retirement/time rechecks; borrower and unprivileged-staff denials; forged owner/status; login/logout; search and bad input; CSRF and GET denials; and the integrated request/approval workflow.

The full Week 3 suite adds six original inventory tests and a concurrent test running ten two-connection rounds. All 21 tests passed in 2.127 seconds in the recorded run. Each concurrency round produced one approval and one conflict. This finite test supports the local prototype behavior, not proof for all deployments.

Incomplete work and changes

The Django/SQLite architecture is retained. New borrower-facing templates and a service layer connect the original inventory component to reservations. Root URL now opens the catalog rather than redirecting to admin; /admin/ remains available. No new dependencies were required.

Cancellation, checkout, return, overdue detection and usage reporting remain planned. Login is implemented for administrator-created accounts; registration and university SSO remain out of scope. No NFR performance benchmark, Safari compatibility run, or full accessibility audit has been completed.

Reference and assistance

Django Software Foundation. n.d. “Databases.” Django Documentation. Accessed October 27, 2026. https://docs.djangoproject.com/en/5.1/ref/databases/.

Implementation, test execution and report preparation used OpenAI assistant support. Student review, explanation of the rules, and an independent browser demonstration are still needed before submission.

Integration and Progress

Representative end-to-end workflow

Borrower login -> catalog and interval search -> request form -> Django validation -> transactional service -> SQLite Pending reservation -> authorized reviewer approval -> persisted Approved status -> borrower status page and updated availability. This is a functioning initial reservation workflow; it does not include physical checkout or return.

Components and integration

Django authentication/session middleware identifies users. Forms convert local browser dates to timezone-aware values and reject invalid inputs. Views enforce route-level authorization, services enforce business rules, ORM models persist relationships, and templates display status and errors. SQLite IMMEDIATE transactions serialize conflicting approvals.

The existing Equipment admin and new Reservation model share the same database. Deactivating equipment retains history and blocks new requests/approval. Reviewer identity and review time are recorded. Reservation foreign keys use PROTECT to preserve linked records.

Start the application

From the package root, create/activate a Python 3.12 virtual environment and install src/campus_checkout/requirements.txt. Then change to src/campus_checkout and run: python manage.py migrate; python manage.py createsuperuser; python manage.py create_demo_borrower; python manage.py runserver. Execute each command separately.

Choose your own passwords. The demo-borrower command creates a fictional borrower and demo laptop. It refuses to overwrite an existing account. Open http://127.0.0.1:8000/. The administrator account can manage inventory at /admin/ and review requests at /review/. Full commands are in START_WEEKS_2_3.md.

Integration and Progress / continued

Manual demonstration to perform

1. Log in as demo-borrower. Search Laptop with a start tomorrow and an end one hour later. Click Request and enter that interval; submit. My requests displays Pending.

2. Log out and log in as your administrator. Open Review requests and approve the request. Log out, return as demo-borrower, and confirm Approved in My requests. Search the same interval and confirm Unavailable.

3. Submit an end before the start; confirm the visible validation message and no new request. Create another valid request overlapping the approved booking; try approving it as administrator. Confirm the conflict error and unchanged Pending state.

4. A non-superuser reviewer needs the explicit inventory.review_reservation permission assigned through admin Users. A borrower's direct approval POST must return 403. Adjacent approved intervals are allowed.

Executed evidence and persistence

The assistant executed manage.py demo_workflow on a separate SQLite database on October 27. It invokes the real Django routes with the Django test client and synthetic accounts. This is HTTP/template integration evidence, not a claimed manual Chrome or Safari session.

Observed: invalid input returned HTTP 200 with a form error and zero created reservations; valid input returned 302 and persisted Pending; borrower approval returned 403; reviewer approval returned 302 and persisted Approved with reviewer identity. Borrower HTML contained Approved, and availability HTML contained Unavailable.

The recorded reservation was #1, equipment EVID-8B4BDAA5, from 2026-10-29 18:26:40 UTC to 19:26:40 UTC. A fresh management-shell process reopened the same database and confirmed Approved plus the borrower/equipment links. This verifies reservation persistence across process restart, not checkout persistence.

Raw evidence: verification/workflow_output.txt, persistence.txt, migrations.txt and provenance.txt. The PermissionDenied traceback in workflow output is the expected logged 403, not a failed demonstration.

Integration and Progress / continued

Tests and requirement status

python manage.py test inventory --noinput --verbosity 2: 21 tests passed. See verification/test_output.txt. The integrated test verifies invalid input, successful request, approval, owner-visible status, and unavailable catalog state; the concurrent test repeats competing approvals ten times. Django system check found no issues; migration drift check found no changes.

Requirement	Status	Scope / remaining work
FR-01	Implemented	Login, logout, credential rejection
FR-02	Partial	Current routes secured; lending pending
FR-03	Implemented	Inventory and inactive item guard
FR-04	Implemented	Search and interval availability
FR-05	Implemented	Validated Pending requests
FR-06	Implemented	Approval/rejection with conflict checks
FR-07	Partial	Own status only; cancellation planned
FR-08 to 10	Planned	Checkout, return, overdue
FR-11	Partial	Reservation persistence; loans pending
FR-12	Planned	Usage counts by date range

Integration and Progress / continued

Design changes and unresolved issues

New templates and transactional services extend the original monolithic Django design. File-backed SQLite testing and IMMEDIATE write transactions address concurrent approvals. No production deployment claim is made. Validation errors are displayed inline or through Django messages.

Unresolved: no lending lifecycle, cancellation, measured NFR-03 workload performance, real-browser compatibility evidence, or complete keyboard/accessibility review. Availability describes approved time reservations, not physical possession of equipment after a late return.

The remaining prototype uses DEBUG and a public local development secret fallback. Deployment requires separate security/configuration work. Synthetic demonstration accounts contain no real student data.

Next five development tasks

1. Add owner-only cancellation with state rules and negative authorization tests. 2. Add checkout records and prevent duplicate/open-item checkout, including late returns. 3. Add idempotent return and overdue queries. 4. Add date-range usage counts verified against a hand-counted fixture. 5. Run workload/browser/accessibility evaluation and prepare the final demonstration and report.

Project Completion Plan

Project Completion Plan

Current delivered scope is an initial reservation application. The work in these reports was implemented and tested with assistant support on October 27, 2026. Student review and independent execution are pending. No fabricated weekly commit history or prior execution is claimed.

Remaining tasks: cancellation; checkout and return lifecycle; overdue queries and usage summary; performance and browser/accessibility evaluation; final documentation and demonstration. Prioritize integrity and permissions before additional UI polish.

Additional data: fictional equipment, borrower accounts, overdue/returned loans, boundary bookings, and a hand-counted reporting fixture. The planned performance dataset is 200 equipment items and 2,000 reservations with 100 sequential authenticated availability requests after warm-up. It has not been benchmarked yet.

Expected computational effort: local Django/SQLite development and small automated test datasets on the Mac. The current 21-test suite took 2.127 seconds; no runtime estimate is asserted for the unimplemented workload evaluation. No GPU, paid API or external student data is required.

Report preparation: map each remaining requirement to code and executed evidence, record actual dates and limitations, use Turabian/Chicago author-date citations where references are used, and rehearse the browser demonstration. Preserve Step 1 and earlier milestone documents.

Submission note: the assignment gives software milestone filenames but its closing paragraph lists research section titles. This review copy uses the software titles consistent with the implemented project; confirm the expected final headings with Professor Pitts before submitting. The earlier Week 1 report is included unchanged as a historical milestone; use the latest run guide for the current catalog workflow.